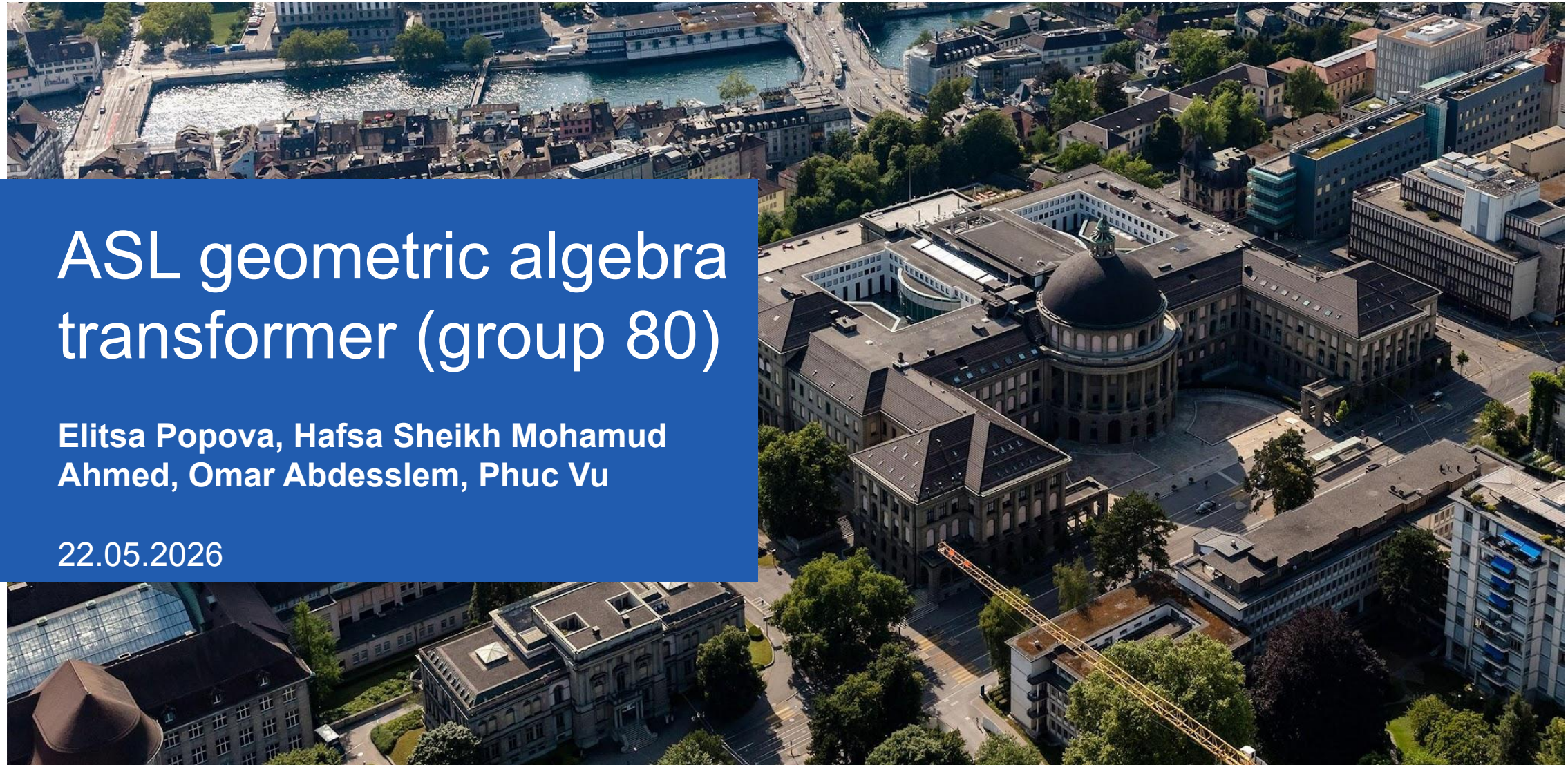
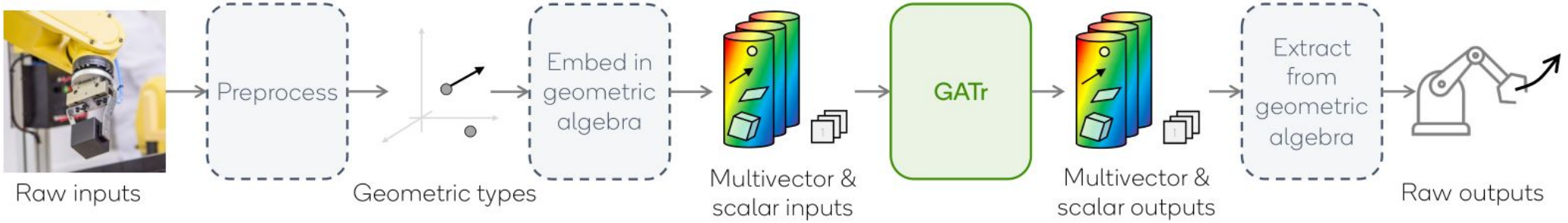


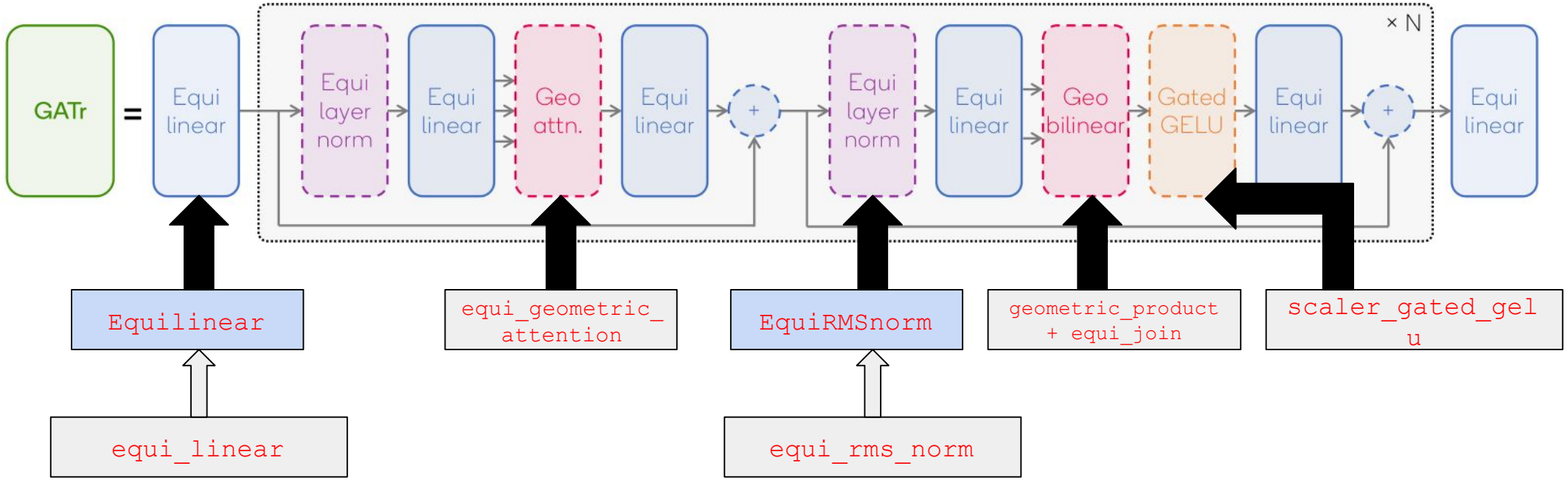
ASL geometric algebra transformer (group 80)

Elitsa Popova, Hafsa Sheikh Mohamud Ahmed, Omar Abdesslem, Phuc Vu

22.05.2026

Algorithm overview





Benchmark is the forward pass of **MVOnlyGATrModel**
fixed config: **B=8**, **N=256**, **channels=2**, **multivector dim 16**

Overview of Changes

- Fixed our Validation
- Fully Switched to ARM
- Implemented Profiling
- Implemented Basic Optimizations
- (Re) Computed Timing Results for Baseline, Optimized, and Reference (Python)
- Plotted Roofline Plots

Validation

- Fixed high margin of error
- Switched to MAPE
- Validation done on both baseline and optimized functions

Validation Results

```
========================================
  EzGATr End-to-End Validation Test
========================================

[INFO] Comparing Python EzGATr with ASL/C++ EzGATr
[INFO] If needed, run: python3 -m ezgatr_extensions.make_basis

[Passed] (B=1, T=4, C=2) MAPE=0.0000%
[Passed] (B=1, T=8, C=2) MAPE=0.0001%
[Passed] (B=1, T=16, C=2) MAPE=0.0001%
[Passed] (B=2, T=4, C=2) MAPE=0.0000%
[Passed] (B=2, T=16, C=2) MAPE=0.0001%
[Passed] (B=4, T=32, C=2) MAPE=0.0001%
[Passed] (B=8, T=64, C=2) MAPE=0.0001%
[Passed] (B=8, T=128, C=2) MAPE=0.0001%
[Passed] (B=8, T=256, C=2) MAPE=0.0001%
[Passed] (B=1, T=4, C=1) MAPE=0.0001%

========================================
  Validation completed successfully
========================================
```

Optimized

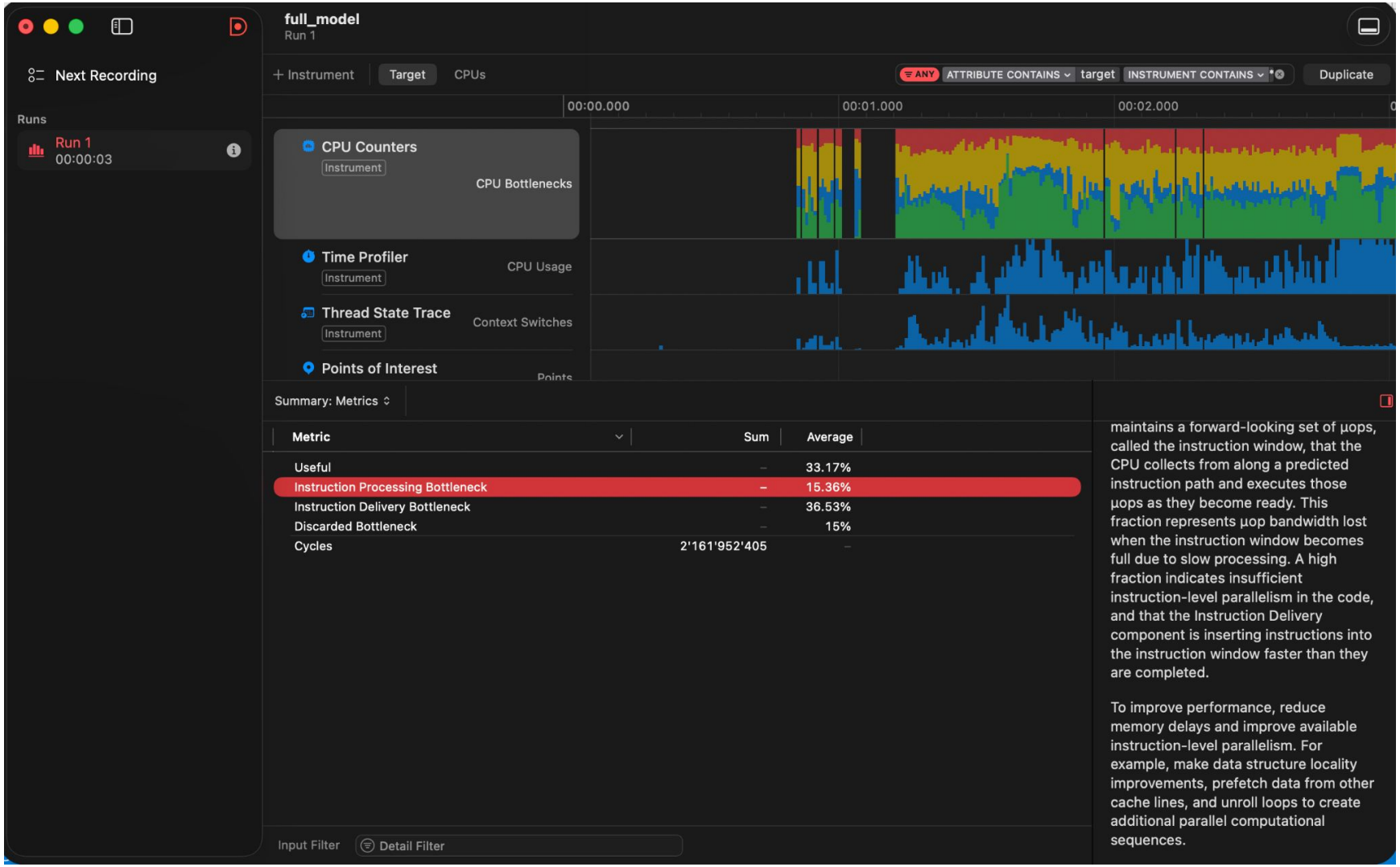
```
========================================
  EzGATr End-to-End Validation Test
========================================

[INFO] Comparing Python EzGATr with ASL/C++ EzGATr
[INFO] If needed, run: python3 -m ezgatr_extensions.make_basis

[Passed] (B=1, T=4, C=2) MAPE=0.0002%
[Passed] (B=1, T=8, C=2) MAPE=0.0001%
[Passed] (B=1, T=16, C=2) MAPE=0.0001%
[Passed] (B=2, T=4, C=2) MAPE=0.0001%
[Passed] (B=2, T=16, C=2) MAPE=0.0001%
[Passed] (B=4, T=32, C=2) MAPE=0.0001%
[Passed] (B=8, T=64, C=2) MAPE=0.0001%
[Passed] (B=8, T=128, C=2) MAPE=0.0001%
[Passed] (B=8, T=256, C=2) MAPE=0.0001%
[Passed] (B=1, T=4, C=1) MAPE=0.0001%

========================================
  Validation completed successfully
========================================
```

Baseline

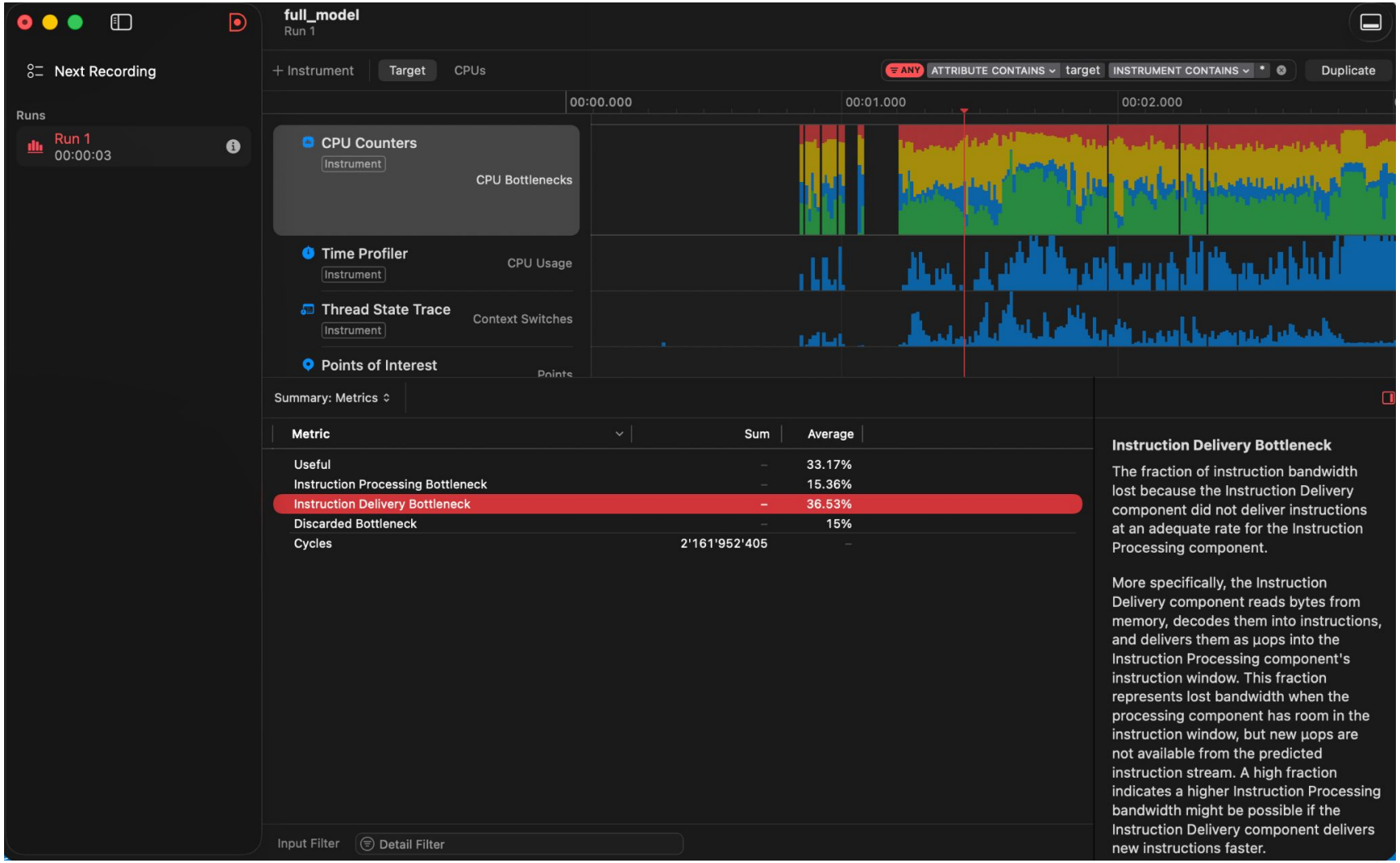
Profiling

- We used the Instruments App on Apple Silicon on XCode
- Ran it 100 times
- Outputted the log and .trace

Profiling: Instruction Processing Bottleneck



Profiling: Instruction Delivery Bottleneck



Bottleneck analysis/profiling (AMD)

We measured cache misses for each of the modules/functions separately to see which ones have the most cache misses. here sorted from most to least number of cache misses:

1 equi_geometric_attention

2. equi_join

3. geometric_product

4. asl_equi_linear

5. equi_rms_norm

6. equi_dual

Profiling: Takeaway

- To improve performance, reduce memory delays and improve available instruction-level parallelism. For example, make data structure locality improvements, prefetch data from other cache lines, and unroll loops to create additional parallel computational sequences.

- To improve performance, reduce instruction memory delays by improving the locality of hot functions, and inline, unroll, and straighten common code sequences to create longer streams of sequentially fetched instructions. For less common code, optimize for smaller code size to improve cache performance using -Os with C-based languages.

- For cache, our matrix optimizations naturally lead to less cache misses.

Optimizations

equi_linear :

- sparsity exploitation (hardcode the ~24 non-zero positions instead of looping over all 9×16×16 entries)
- accumulators
- move out pointer arithmetic out of inner loops
- after normalization every non-zero of the basis is one of 4 values, move basis multiplication out of inner loop

Optimizations

equi_geometric_attention:

- unrolls the inner loop into 16 FMAs
- reads the original token data directly and computes the similarity on the fly
- DAA einsum replaced by 5 scalar terms
- move trivector normalize out of inner loop
- branch out on zero softmax weights and fully-masked rows (sparsity)
- IPA dot skips the 4 always-zero blades (sparsity exploit)
- collapse the stages into one kernel (don’t build of q_cat or k_cat -> no allocation of memory and better cache behavior)

Optimizations

geometric_product&outer_product:

(We used generated kernels, hardcoding the sparsity instead of using nested loops. This makes the function much faster both in terms of memory usage and the amount of floating point operations.) We also applied more optimizations, to further bring down the runtime like using FMAs.
Example(geometric_product):We refrained from further unrolling with Accumulators, because the amount of parallel computation in the generated kernel function was already ideal for M3 Apple Silicon.

```
for (int64_t b = 0; b < batch_size; ++b) {
    int b16 = b * 16;
    gp_kernel_one_batch(x_ptr + b16, y_ptr + b16, result_ptr + b16);
}
```

Optimizations

`equi_geometric_product :`

Replaced 16x16x16 first with CSR, then changed to hard-coded kernel that hardcodes only non-zero contractions, only 192 additions/substractions for 16 output blades instead of 16x16x16 multiplications. Got rid of zero multiplications, tensor loops and basis.

`equi_join :`

Same as geometric product, hardcodes sign flips, only 81 additions/substractions for the 16 output blades instead of 16x16x16 multiplications.

Optimizations:

- equi_rms_norm_kernel:
- C++ function that normalizes each multivector channel by its equivariant RMS norm.

  The selected blades never change, Only 8 of the 16 multivector blades contribute to the norm:

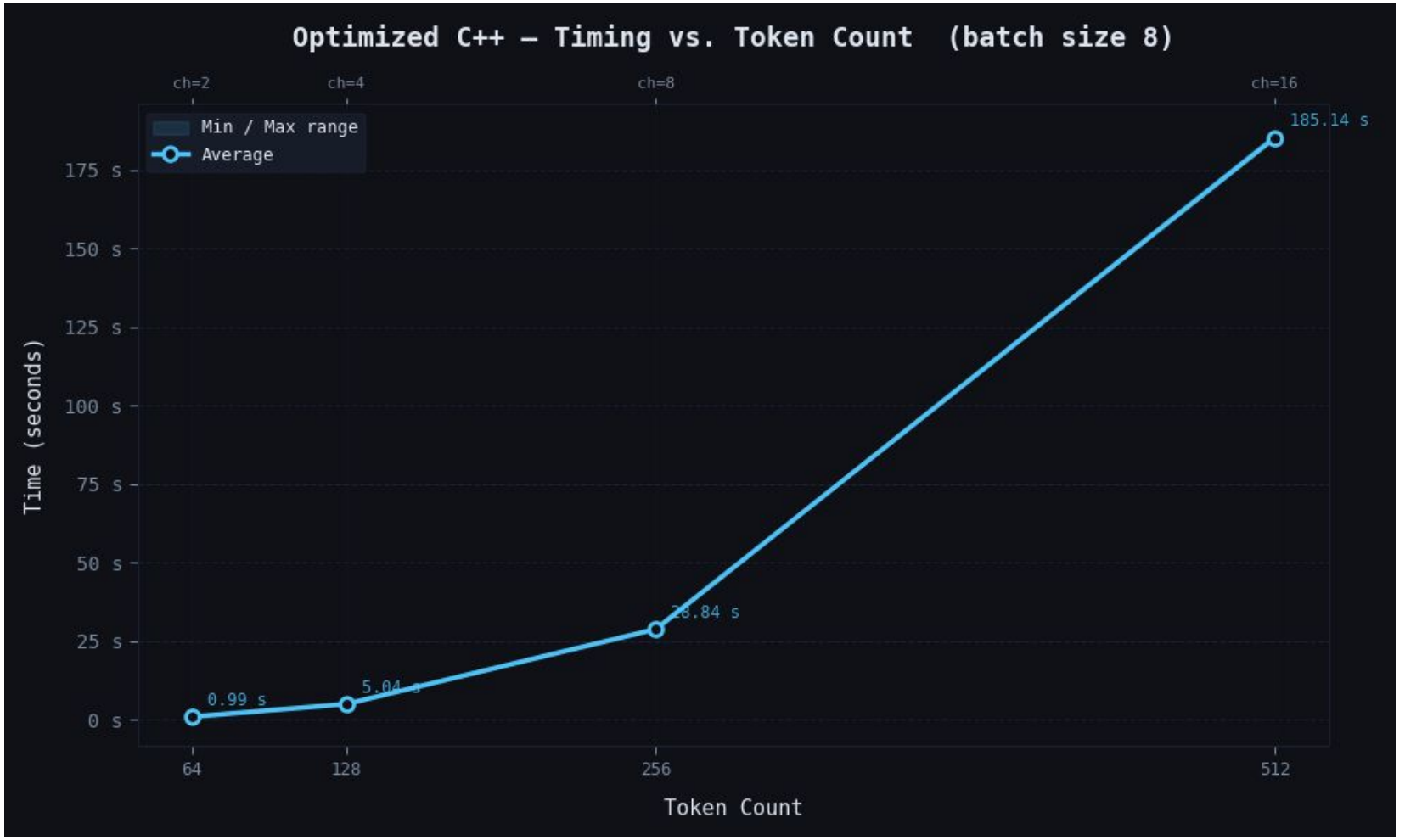
  0, 2, 3, 4, 8, 9, 10, 14

- Hardcoding them removes: the full loop from 0 to 15, and the indirect memory access through the table
  - x[0] * x[0] +
  - x[2] * x[2] +
  - x[3] * x[3] +
  - ...
  - x[14] * x[14]

- Method Applied: Sparse selector hardcoding / loop unrolling. | Speed= 2x

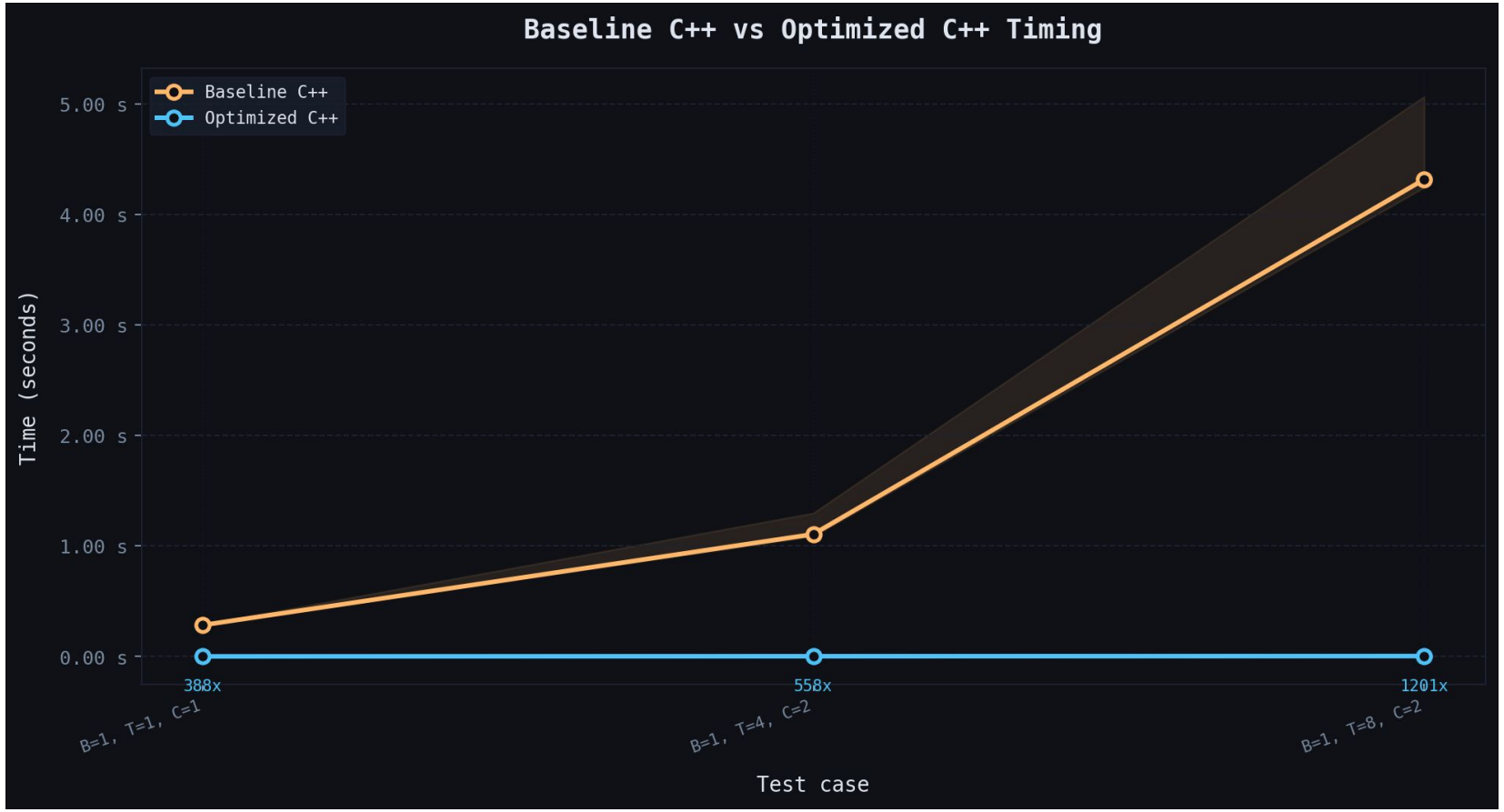
Timing Results after optimizations

After optimizations, we had **400x Speedup**



Timing Results after optimizations

Initial comparison, only three points from baseline because it takes ages



Per function performance

BASELINE

```
[STEP] B=8, T=64, C=2
[geometric_product] avg=2.5348 ms | min=2.4828 ms | max=2.9297 ms
[equi_linear] avg=3.0484 ms | min=3.0382 ms | max=3.0831 ms
[equi_join] avg=4.6508 ms | min=4.6348 ms | max=4.7702 ms
[equi_rms_norm] avg=0.0069 ms | min=0.0067 ms | max=0.0074 ms
[scaler_gated_gelu] avg=0.0075 ms | min=0.0074 ms | max=0.0078 ms
[equi_geometric_attention] avg=2.2509 ms | min=2.2385 ms | max=2.2851 ms

[STEP] B=8, T=128, C=4
[geometric_product] avg=9.9698 ms | min=9.9306 ms | max=10.0287 ms
[equi_linear] avg=30.9100 ms | min=30.0380 ms | max=33.4855 ms
[equi_join] avg=13.3756 ms | min=12.9292 ms | max=13.6780 ms
[equi_rms_norm] avg=0.0262 ms | min=0.0257 ms | max=0.0292 ms
[scaler_gated_gelu] avg=0.0291 ms | min=0.0282 ms | max=0.0342 ms
[equi_geometric_attention] avg=24.7417 ms | min=15.5471 ms | max=248.8651 ms

[STEP] B=8, T=256, C=8
[geometric_product] avg=42.7635 ms | min=39.2605 ms | max=60.7027 ms
[equi_linear] avg=272.7963 ms | min=259.6503 ms | max=320.4120 ms
[equi_join] avg=46.5484 ms | min=45.0587 ms | max=47.5187 ms
[equi_rms_norm] avg=0.0770 ms | min=0.0758 ms | max=0.0777 ms
[scaler_gated_gelu] avg=0.1179 ms | min=0.1171 ms | max=0.1187 ms
[equi_geometric_attention] avg=137.8232 ms | min=135.5122 ms | max=157.9398 ms

[STEP] B=8, T=512, C=16
[geometric_product] avg=160.9239 ms | min=154.9998 ms | max=183.5474 ms
[equi_linear] avg=2268.5176 ms | min=2166.6575 ms | max=2497.2429 ms
[equi_join] avg=187.6760 ms | min=183.1438 ms | max=191.9271 ms
[equi_rms_norm] avg=0.3866 ms | min=0.3522 ms | max=0.4251 ms
[scaler_gated_gelu] avg=0.6775 ms | min=0.6410 ms | max=0.7175 ms
[equi_geometric_attention] avg=1218.2382 ms | min=1187.5501 ms | max=1493.3894 ms
```

OPTIMIZED

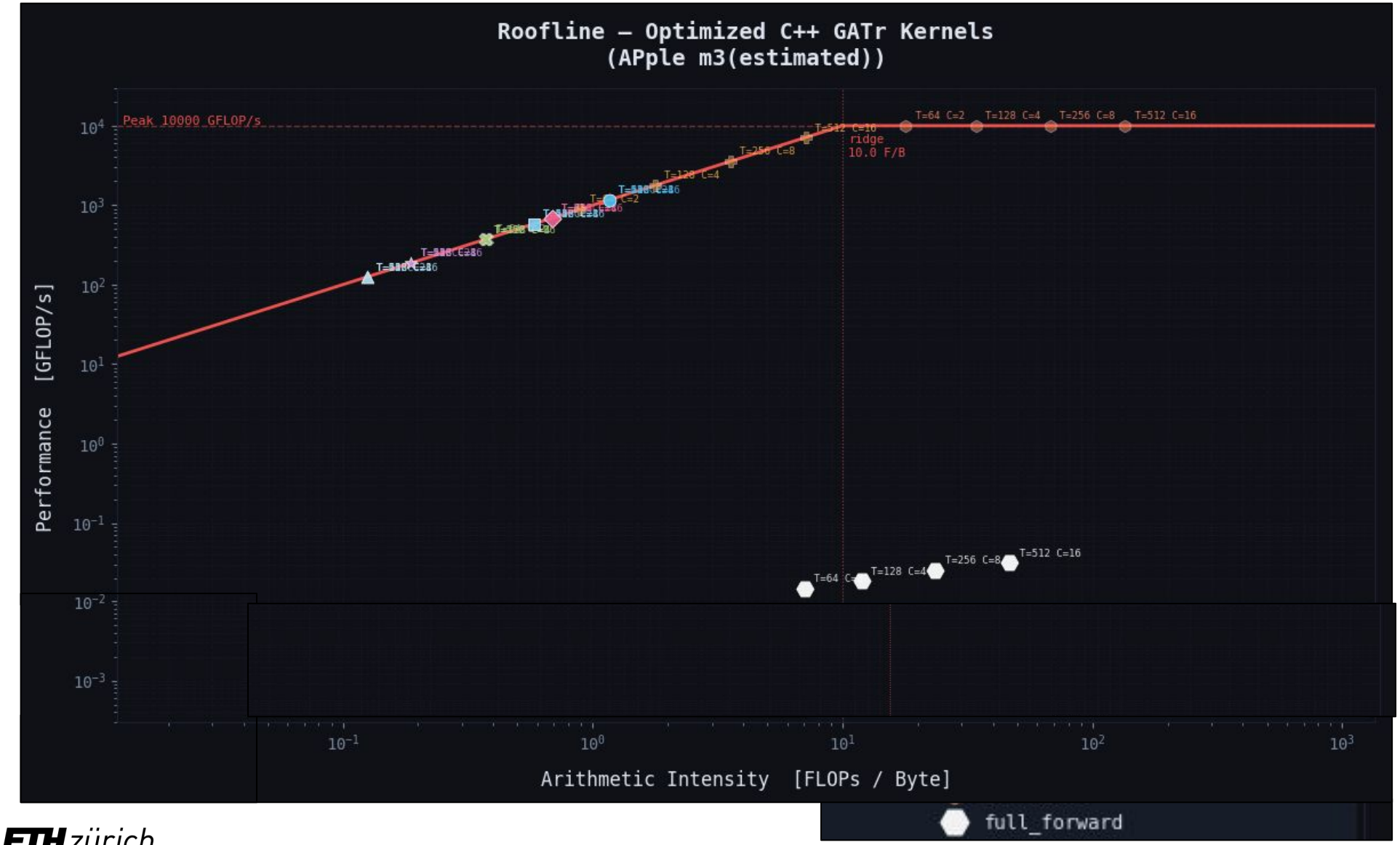
```
[geometric_product] avg=0.0521 ms | min=0.0488 ms | max=0.0560 ms
[equi_linear] avg=0.0237 ms | min=0.0227 ms | max=0.0355 ms
[equi_join] avg=0.0137 ms | min=0.0134 ms | max=0.0141 ms
[equi_rms_norm] avg=0.0083 ms | min=0.0082 ms | max=0.0088 ms
[scaler_gated_gelu] avg=0.0087 ms | min=0.0085 ms | max=0.0090 ms

[STEP] B=8, T=128, C=4
[geometric_product] avg=0.1336 ms | min=0.1215 ms | max=0.1487 ms
[equi_linear] avg=0.1321 ms | min=0.1197 ms | max=0.1353 ms
[equi_join] avg=0.0713 ms | min=0.0693 ms | max=0.0811 ms
[equi_rms_norm] avg=0.0413 ms | min=0.0408 ms | max=0.0500 ms
[scaler_gated_gelu] avg=0.0430 ms | min=0.0424 ms | max=0.0435 ms

[STEP] B=8, T=256, C=8
[geometric_product] avg=0.2831 ms | min=0.2797 ms | max=0.2884 ms
[equi_linear] avg=0.6370 ms | min=0.6352 ms | max=0.6450 ms
[equi_join] avg=0.1661 ms | min=0.1621 ms | max=0.1764 ms
[equi_rms_norm] avg=0.0861 ms | min=0.0836 ms | max=0.0934 ms
[scaler_gated_gelu] avg=0.1111 ms | min=0.1104 ms | max=0.1123 ms

[STEP] B=8, T=512, C=16
[geometric_product] avg=1.2629 ms | min=1.2311 ms | max=1.3236 ms
[equi_linear] avg=5.5025 ms | min=5.3225 ms | max=7.5530 ms
[equi_join] avg=0.8975 ms | min=0.8109 ms | max=1.1387 ms
[equi_rms_norm] avg=0.3904 ms | min=0.3643 ms | max=0.4655 ms
[scaler_gated_gelu] avg=0.6666 ms | min=0.6195 ms | max=0.8053 ms
```

Speed-ups

Config	Op	Before (ms)	After (ms)	Speedup
T=128, C=4	geometric_product	9.9698	0.1336	74.6x
T=128, C=4	equi_linear	30.9100	0.1321	234.0x
T=128, C=4	equi_join	13.3756	0.0713	187.6x
T=256, C=8	geometric_product	42.7635	0.2831	151.1x
T=256, C=8	equi_linear	272.7963	0.6370	428.3x
T=256, C=8	equi_join	46.5484	0.1661	280.2x
T=512, C=16	geometric_product	160.9239	1.2629	127.4x
T=512, C=16	equi_linear	2268.5176	5.5025	412.3x
T=512, C=16	equi_join	187.6760	0.8975	209.1x

Also found considerable speed-up for geometric-attention but forgot to include it

Roofline Plot



Summary of Changes

- Fixed our Validation
- Fully Switched to ARM
- Implemented Profiling
- Implemented Basic Optimizations
- (Re) Computed Timing Results for Baseline, and Optimized.
- Plotted Roofline Plots

Thanks for joining !

Extras

Reference Python vs C++ code: